

Retrieval-Augmented Generation (RAG) Machine Learning Knowledge Assistant

1. Introduction

This project implements an end-to-end Retrieval-Augmented Generation (RAG) system using a Machine Learning book in PDF format as the knowledge source. The objective is to build an intelligent assistant capable of answering technical questions related to Machine Learning and Deep Learning by retrieving relevant content from the document and generating accurate responses.

The system integrates document preprocessing, semantic search using embeddings, vector database retrieval, and large language model (LLM)-based answer generation. This approach ensures that responses are context-aware, accurate, and grounded in the source document, thereby minimizing hallucinations.

Part 1: Data Understanding and Preprocessing

Document Loading and Text Extraction

- The dataset used is a PDF version of a Machine Learning book. The document was processed using the PyMuPDF library to extract text from all pages.
- Total number of pages: 392
- Text was extracted page-wise and stored for further processing.

Document Structure Exploration

During exploration, the following observations were made:

- Some pages contained headers and footers repeated across pages
- A few pages had less textual content, such as chapter separators or image-heavy pages
- Text extraction quality was generally clean and readable with minimal noise
- No major formatting issues were observed*

These observations helped guide preprocessing and chunking decisions.

Text Chunking Strategy

To prepare the document for embedding and retrieval, the extracted text was split into smaller chunks.

Chunk size: 800 characters

Overlap: 100 characters

Chunking breaks large text into manageable semantic units. Overlap ensures that contextual information is preserved across chunk boundaries and prevents loss of meaning between adjacent chunks.

Total number of chunks generated:1182

Part 2: Embedding and Vector Database

Embedding Model

Embeddings were generated using SentenceTransformers with the model “all-MiniLM-L6-v2”.

Reasons for selection:

- Lightweight and computationally efficient
- Provides strong semantic similarity performance
- Suitable for real-time applications
- Performs well on CPU-based environments

Vector Database

The embeddings were stored using FAISS (Facebook AI Similarity Search).

Key characteristics:

- Stores dense vector representations of text
- Enables efficient nearest-neighbor search
- Uses L2 distance for similarity comparison
- Scales well for large datasets

Example Retrieval

For the query “What is overfitting?”, the system retrieves the most relevant chunks from the document. These chunks contain definitions and explanations directly related to overfitting, demonstrating effective semantic search.

Part 3: Retrieval Pipeline

The retrieval pipeline operates as follows:

1. The user inputs a query
2. The query is converted into an embedding
3. The FAISS index is searched
4. The top-k most relevant chunks are retrieved

Experimentation with k Values

Different values of k were tested to evaluate retrieval performance.

- k = 3
Highly precise results but may miss supporting context
- k = 5
Balanced results with sufficient context and relevance
- k = 7
More comprehensive results but may introduce some irrelevant information

Conclusion:

A value of k = 5 provided the best balance between precision and context.

Part 4: Answer Generation (RAG)

Language Model

The system uses the Gemini 2.5 Flash model from Google for answer generation.

Prompt Design

The prompt was designed to ensure grounded and reliable responses. The model is instructed to answer only using the provided context. If the answer is not present in the retrieved content, the model responds with “Not found in document”.

Importance of Prompt Design:

- Reduces hallucination
- Ensures answers are based on retrieved content
- Improves reliability and trustworthiness

Grounded Answer Example

Query: What is overfitting?

Response with context:

QUESTION: What is overfitting?

ANSWER:

Building a model that is too complex for the amount of information available is called overfitting. Overfitting occurs when you fit a model too closely to the particularities of the training set and obtain a model that works well on the training set but is not able to generalize well to new data.

RETRIEVED CONTEXT:

--- Chunk 1 ---

```
X_test, y_test)))
```

```
Out[28]:
```

```
Training set score: 0.67
```

```
Test set score: 0.66
```

An R2 of around 0.66 is not very good, but we can see that the scores on the training and test sets are very close together. This means we are likely underfitting, not overfitting. For this one-dimensional dataset, there is little danger of overfitting, as the model is very simple (or restricted). However, with higher-dimensional datasets (meaning datasets with a large number of features), linear models become more powerful

--- Chunk 2 ---

ed” part of the rule hinges
on a single customer.

The only measure of whether an algorithm will perform well on new data is the evaluation on the test set. However, intuitively³ we expect simple models to generalize better to new data. If the rule was “People older than 50 want to buy a boat,” and this would explain the behavior of all the customers, we would trust it more than the rule involving children and marital status in addition to age. Therefore, we always want to find the simplest mod

--- Chunk 3 ---

f the new data point falls into. The output for this data point is the mean target of the training points in this leaf.

Controlling complexity of decision trees

Typically, building a tree as described here and continuing until all leaves are pure leads to models that are very complex and highly overfit to the training data. The presence of pure leaves mean that a tree is 100% accurate on the training set; each data point in the training set is in a leaf that has the correct majority class. The o

--- Chunk 4 ---

people who are 66, 52, 53, or
Generalization, Overfitting, and Underfitting

|

27

--- Chunk 5 ---

trip predicted as class 0 around the point belonging to class 0
to the very right. This is not how one would imagine the decision boundary to look, and the decision boundary focuses a lot on single outlier points that are far away from the other points in that class.

There are two common strategies to prevent overfitting: stopping the creation of the tree early (also called pre-pruning), or building the tree but then removing or collapsing nodes that contain little information (also called pos

Observation:

The generated answer closely aligns with the retrieved content, demonstrating that the system produces grounded and accurate responses.

Part 5: End-to-End RAG Application

System Integration

The final system integrates all components:

- PDF text extraction
- Text chunking
- Embedding generation
- Vector storage using FAISS
- Retrieval pipeline
- LLM-based answer generation

Real-Time Question Answering

The system supports real-time queries such as:

- What is overfitting?
- What is a neural network?
- Difference between classification and regression

The system retrieves relevant content and generates accurate answers dynamically.

Challenges and Improvements

Challenges Faced

- Selecting optimal chunk size
- Ensuring relevant retrieval results
- Minimizing hallucination in generated responses

Possible Improvements

- Hybrid search combining keyword-based and semantic search
- Metadata filtering based on chapters or sections

- Adding source citations such as page numbers
- Re-ranking retrieved results for improved accuracy

Conclusion

This project successfully demonstrates the implementation of a Retrieval-Augmented Generation system for a Machine Learning knowledge assistant.

The system provides a scalable approach for building domain-specific AI assistants using unstructured documents. The RAG architecture is highly effective for knowledge retrieval systems, document-based question answering, and enterprise AI applications. This implementation highlights how combining retrieval with generation leads to more reliable and explainable AI systems.